

Actividad #3

Diego Armando Porto Ramos

Correo: dporto@unal.edu.co

C.C. 1039474932

Programación Orientada a Objetos

Profesor:

Walter Hugo Arboleda

Universidad Nacional de Colombia

Sede Medellín

2026

Repositorio de GitHub:

https://github.com/DiegoPortoR/Programacion-Orientada-a-Objetos/tree/main/Entregas/Ejercicios_Actividad_3

Ejercicio #1 - Ejercicio 8.2

Se requiere desarrollar un programa con interfaz gráfica de usuario que genere una ventana donde se solicite el ingreso de cinco notas obtenidas por un estudiante.

El programa debe calcular y mostrar en la parte inferior de la ventana los siguientes datos:

- ▶ El promedio de notas ingresadas.
- ▶ La desviación estándar de las notas ingresadas.
- ▶ La mayor nota obtenida.
- ▶ La menor nota obtenida.

```
Ejercicio_8_2.py X
Entregas > Ejercicios_Actividad_3 > Ejercicio_8_2.py > EstadisticaNotas > calcular_desviacion

1  import tkinter as tk
2  from tkinter import ttk, messagebox
3
4  # =====
5  # LÓGICA DE NEGOCIO / PROCESAMIENTO DE DATOS
6  # =====
7  class EstadisticaNotas:
8      def __init__(self):
9          # Inicializamos con una lista vacía para almacenar los floats
10         self.calificaciones = []
11
12     def registrar_calificaciones(self, *notas_texto):
13         """Procesa, limpia y valida las notas ingresadas desde la GUI."""
14         try:
15             # Convertimos todas las entradas a float quitando espacios en un solo paso
16             self.calificaciones = [float(nota.strip()) for nota in notas_texto]
17         except ValueError:
18             raise ValueError("Por favor, asegúrate de ingresar únicamente valores numéricos.")
19
20     def obtener_promedio(self):
21         if not self.calificaciones:
22             return 0.0
23         return sum(self.calificaciones) / len(self.calificaciones)
24
25     def obtener_maximo(self):
26         return max(self.calificaciones) if self.calificaciones else 0.0
27
```

```

27
28     def obtener_minimo(self):
29         return min(self.calificaciones) if self.calificaciones else 0.0
30
31     def calcular_desviacion(self):
32         if not self.calificaciones:
33             return 0.0
34
35         media = self.obtener_promedio()
36         # Calculamos la varianza de forma más directa y pythonica
37         varianza = sum((nota - media) ** 2 for nota in self.calificaciones) / len(self.calificaciones)
38         # Retornamos la raíz cuadrada de la varianza (Desviación estándar poblacional)
39         return varianza ** 0.5
40

```

```

42 # =====
43 # INTERFAZ GRÁFICA DE USUARIO (GUI)
44 # =====
45 class VentanaNotasApp:
46     def __init__(self, ventana_principal):
47         self.root = ventana_principal
48         self.root.title("Gestor Estadístico de Calificaciones")
49         self.root.resizable(False, False)
50
51         # Instancia del motor de cálculo
52         self.procesador = EstadisticaNotas()
53         self.campos_entrada = []
54
55         # --- SECCIÓN DE ENTRADA DE DATOS ---
56         seccion_entrada = ttk.LabelFrame(ventana_principal, text=" Registro de Notas ", padding=15)
57         seccion_entrada.pack(padx=15, pady=10, fill="x")
58
59         for idx in range(5):
60             lbl = ttk.Label(seccion_entrada, text=f"Calificación {idx + 1}:")
61             lbl.grid(row=idx, column=0, padx=8, pady=6, sticky="e")
62
63             txt_entry = ttk.Entry(seccion_entrada, width=12)
64             txt_entry.grid(row=idx, column=1, padx=8, pady=6)
65             txt_entry.insert(0, "0.0")
66
67             self.campos_entrada.append(txt_entry)
68

```

```

68
69         # --- BOTÓN DE ACCIÓN ---
70         self.btn_procesar = ttk.Button(
71             ventana_principal,
72             text="Calcular Métricas",
73             command=self.ejecutar_calculos
74         )
75         self.btn_procesar.pack(pady=10)
76
77         # --- SECCIÓN DE RESULTADOS ---
78         seccion_resultados = ttk.LabelFrame(ventana_principal, text=" Métricas Calculadas ", padding=15)
79         seccion_resultados.pack(padx=15, pady=10, fill="x")
80
81         # Usamos un diccionario para gestionar las etiquetas de salida de forma limpia
82         self.indicadores = {}
83         metricas = [
84             ("promedio", "Promedio General:"),
85             ("maximo", "Nota Más Alta:"),
86             ("minimo", "Nota Más Baja:"),
87             ("desviacion", "Desviación Estándar:")
88         ]
89

```

```

89
90     for posicion, (clave, texto) in enumerate(metricas):
91         lbl_titulo = ttk.Label(seccion_resultados, text=texto, font=("Helvetica", 9, "bold"))
92         lbl_titulo.grid(row=posicion, column=0, sticky="w", pady=4, padx=5)
93
94         lbl_valor = ttk.Label(seccion_resultados, text="-")
95         lbl_valor.grid(row=posicion, column=1, sticky="w", pady=4, padx=5)
96
97         self.indicadores[clave] = lbl_valor
98
99     def ejecutar_calculos(self):
100         """Pone en comunicación la interfaz con la lógica de negocio."""
101         try:
102             # Extraemos los textos de las cajas usando list comprehension
103             valores_ingresados = [campo.get() for campo in self.campos_entrada]
104
105             # Enviamos los datos a la clase lógica
106             self.procesador.registrar_calificaciones(*valores_ingresados)
107
108             # Realizamos los cálculos correspondientes
109             media = self.procesador.obtener_promedio()
110             nota_max = self.procesador.obtener_maximo()
111             nota_min = self.procesador.obtener_minimo()
112             desv_est = self.procesador.calcular_desviacion()
113

```

```

113
114         # Actualizamos los elementos visuales con los nuevos formatos
115         self.indicadores["promedio"].config(text=f"{media:.2f}")
116         self.indicadores["maximo"].config(text=f"{nota_max:.1f}")
117         self.indicadores["minimo"].config(text=f"{nota_min:.1f}")
118         self.indicadores["desviacion"].config(text=f"{desv_est:.3f}")
119
120         except ValueError as error_mensaje:
121             # Alerta flotante en caso de errores de escritura
122             messagebox.showerror("Error en los Datos", str(error_mensaje))
123
124
125     # =====
126     # ARRANQUE DE LA APLICACIÓN
127     # =====
128     if __name__ == "__main__":
129         ventana_raiz = tk.Tk()
130         app = VentanaNotasApp(ventana_raiz)
131         ventana_raiz.mainloop()

```

Gestor Estadí...

Registro de Notas

Calificación 1:

Calificación 2:

Calificación 3:

Calificación 4:

Calificación 5:

Calcular Métricas

Métricas Calculadas

Promedio General:

3.10

Nota Más Alta:

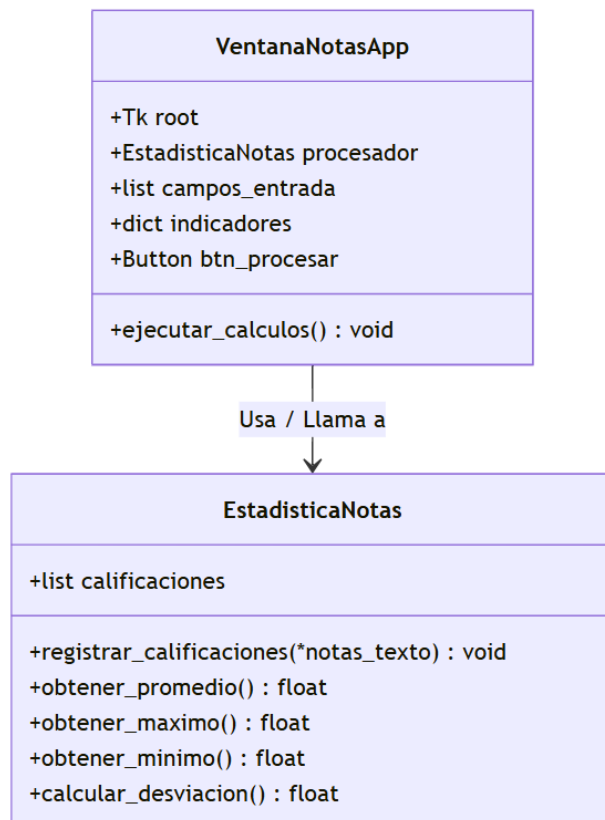
4.6

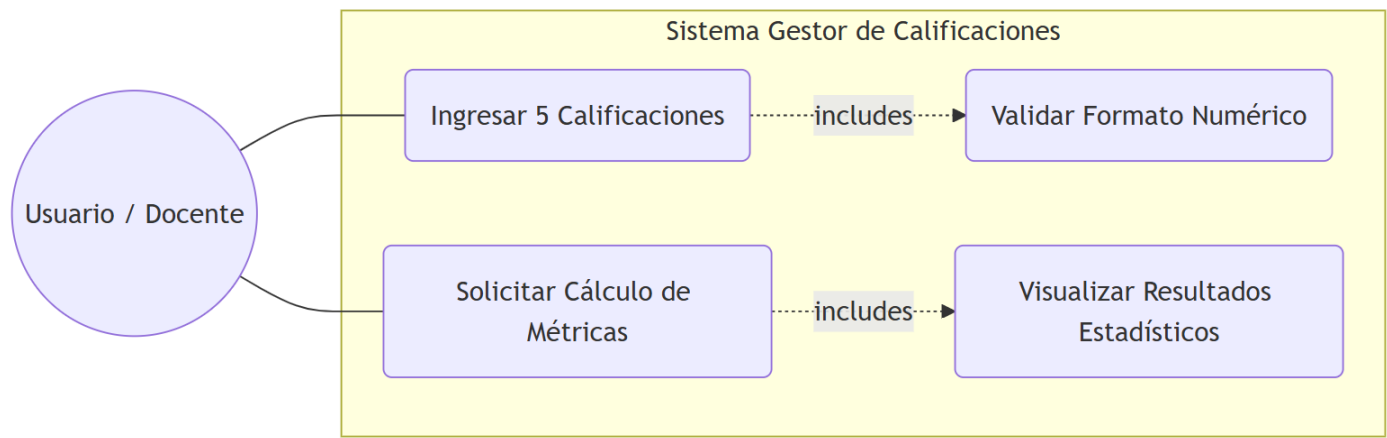
Nota Más Baja:

1.9

Desviación Estándar:

0.949





Ejercicio #2 – Ejercicio 8.3

Se requiere desarrollar un programa con interfaz gráfica de usuario que permita calcular el volumen y superficie de varias figuras geométricas. Las figuras geométricas son el cilindro, la esfera y la pirámide.

- ▶ Para el cilindro se solicitan su radio y altura (en centímetros).
- ▶ Para la esfera, su radio (en centímetros).
- ▶ Para la pirámide, su base, altura y apotema (en centímetros).

Una vez ingresados estos datos, el programa calcula el volumen y superficie de cada figura. Para desarrollar el programa se debe crear una jerarquía de clases para las diferentes figuras geométricas requeridas.

```
1  import tkinter as tk
2  from tkinter import ttk, messagebox
3  from abc import ABC, abstractmethod
4  import math
5
6  # =====
7  # 1. CAPA DE LÓGICA MATEMÁTICA (Jerarquía de Clases - POO)
8  # =====
9
10 class FiguraSolida(ABC):
11     """Clase abstracta pura que sirve de molde para los sólidos."""
12     def __init__(self, nombre: str):
13         self.nombre = nombre
14
15     @abstractmethod
16     def calcular_volumen(self) -> float:
17         pass
18
19     @abstractmethod
20     def calcular_superficie(self) -> float:
21         pass
22
```

```
23
24 class Cilindro(FiguraSolida):
25     def __init__(self, radio: float, altura: float):
26         super().__init__("Cilindro")
27         self.radio = radio
28         self.altura = altura
29
30     def calcular_volumen(self) -> float:
31         return math.pi * (self.radio ** 2) * self.altura
32
33     def calcular_superficie(self) -> float:
34         area_lateral = 2 * math.pi * self.radio * self.altura
35         area_bases = 2 * math.pi * (self.radio ** 2)
36         return area_lateral + area_bases
37
38
39 class Esfera(FiguraSolida):
40     def __init__(self, radio: float):
41         super().__init__("Esfera")
42         self.radio = radio
43
44     def calcular_volumen(self) -> float:
45         return (4 / 3) * math.pi * (self.radio ** 3)
46
47     def calcular_superficie(self) -> float:
48         return 4 * math.pi * (self.radio ** 2)
49
```

```

50
51 class PiramideCuadrada(FiguraSolida):
52     def __init__(self, lado_base: float, altura: float, apotema: float):
53         super().__init__("Pirámide Cuadrada")
54         self.lado_base = lado_base
55         self.altura = altura
56         self.apotema = apotema
57
58     def calcular_volumen(self) -> float:
59         area_base = self.lado_base ** 2
60         return (1 / 3) * area_base * self.altura
61
62     def calcular_superficie(self) -> float:
63         area_base = self.lado_base ** 2
64         perimetro_base = 4 * self.lado_base
65         area_lateral = (perimetro_base * self.apotema) / 2
66         return area_base + area_lateral
67
68
69 # =====
70 # 2. CAPA DE INTERFAZ GRÁFICA DE USUARIO
71 # =====
72

```

```

72
73 class CalculadoraGeometricaApp:
74     def __init__(self, root):
75         self.root = root
76         self.root.title("Calculador Geométrico Pro")
77         self.root.geometry("450x420")
78         self.root.resizable(False, False)
79
80         # Configuración de estilos Tkinter (Temas y Colores)
81         style = ttk.Style()
82         style.theme_use('clam')
83
84         # Personalización de las pestañas
85         style.configure("TNotebook", background=□"#f0f2f5")
86         style.configure("TNotebook.Tab", font=("Arial", 10, "bold"), padding=[12, 4])
87         style.configure("TFrame", background=□"#ffffff")
88         style.configure("TLabelframe", background=□"#ffffff")
89         style.configure("TLabelframe.Label", font=("Arial", 10, "bold"), background=□"#ffffff", foreground=■"#333333")
90         style.configure("TLabel", background=□"#ffffff", font=("Arial", 10))
91
92         # Botón personalizado estilizado
93         style.configure("Accent.TButton", font=("Arial", 10, "bold"), foreground="white", background=■"#007bff")
94         style.map("Accent.TButton", background=[('active', ■"#0056b3")])
95
96         # Contenedor principal de pestañas
97         self.notebook = ttk.Notebook(self.root)
98         self.notebook.pack(expand=True, fill="both", padx=15, pady=15)
99

```

```

100     # Inicialización de pestañas
101     self.crear_pestana_cilindro()
102     self.crear_pestana_esfera()
103     self.crear_pestana_piramide()
104
105     def obtener_valor_valido(self, entrada_widget, nombre_campo) -> float:
106         """Valida que la entrada sea un número float positivo."""
107         try:
108             texto = entrada_widget.get().replace(",", ".").strip()
109             valor = float(texto)
110             if valor <= 0:
111                 raise ValueError
112             return valor
113         except ValueError:
114             raise ValueError(f"El campo '{nombre_campo}' debe ser un número válido mayor a cero.")
115
116     # -----
117     # PESTAÑA: CILINDRO
118     # -----
119     def crear_pestana_cilindro(self):
120         pestana = ttk.Frame(self.notebook, padding=15)
121         self.notebook.add(pestana, text="Cilindro")
122
123

```

```

123     # Texto informativo / Guía
124     lbl_info = ttk.Label(pestana, text="Cálculo basado en el Radio de las bases y su Altura.",
125                          font=("Arial", 9, "italic"), foreground=■ "#666666")
126     lbl_info.grid(row=0, column=0, columnspan=2, sticky="w", pady=(0, 10))
127
128     # Entradas
129     ttk.Label(pestana, text="Radio (r) en cm:").grid(row=1, column=0, sticky="w", pady=6)
130     entry_radio = ttk.Entry(pestana, width=18, font=("Arial", 10))
131     entry_radio.grid(row=1, column=1, pady=6, padx=10, sticky="w")
132     entry_radio.insert(0, "5.0")
133
134     ttk.Label(pestana, text="Altura (h) en cm:").grid(row=2, column=0, sticky="w", pady=6)
135     entry_altura = ttk.Entry(pestana, width=18, font=("Arial", 10))
136     entry_altura.grid(row=2, column=1, pady=6, padx=10, sticky="w")
137     entry_altura.insert(0, "10.0")
138
139     # Cuadro de Resultados Destacados
140     frame_res = ttk.LabelFrame(pestana, text=" Resultados Calculados ", padding=12)
141     frame_res.grid(row=4, column=0, columnspan=2, sticky="ew", pady=(15, 0))
142
143     lbl_res_vol = ttk.Label(frame_res, text="Volumen (V): --", font=("Arial", 11, "bold"), foreground=■ "#0056b3")
144     lbl_res_vol.pack(anchor="w", pady=4)
145     lbl_res_sup = ttk.Label(frame_res, text="Superficie Total (A): --", font=("Arial", 11, "bold"),
146                             foreground=■ "#28a745")
147     lbl_res_sup.pack(anchor="w", pady=4)

```

```

148     def calcular():
149         try:
150             r = self.obtener_valor_valido(entry_radio, "Radio")
151             h = self.obtener_valor_valido(entry_altura, "Altura")
152
153             cilindro = Cilindro(r, h)
154             lbl_res_vol.config(text=f"Volumen (V): {cilindro.calcular_volumen():.2f} cm³")
155             lbl_res_sup.config(text=f"Superficie Total (A): {cilindro.calcular_superficie():.2f} cm²")
156         except ValueError as e:
157             messagebox.showerror("Error de datos", str(e))
158
159         ttk.Button(pestana, text="Calcular Cilindro", style="Accent.TButton", command=calcular).grid(row=3, column=0,
columnspan=2, pady=(15, 5))
160
161     # -----
162     # PESTAÑA: ESFERA
163     # -----
164     def crear_pestana_esfera(self):
165         pestana = ttk.Frame(self.notebook, padding=15)
166         self.notebook.add(pestana, text="Esfera")
167
168         lbl_info = ttk.Label(pestana, text="Cálculo simétrico basado únicamente en el Radio del cuerpo.",
169                             font=("Arial", 9, "italic"), foreground=■ "#666666")
170         lbl_info.grid(row=0, column=0, columnspan=2, sticky="w", pady=(0, 10))
171

```

```

171
172         ttk.Label(pestana, text="Radio (r) en cm:").grid(row=1, column=0, sticky="w", pady=6)
173         entry_radio = ttk.Entry(pestana, width=18, font=("Arial", 10))
174         entry_radio.grid(row=1, column=1, pady=6, padx=10, sticky="w")
175         entry_radio.insert(0, "5.0")
176
177         frame_res = ttk.LabelFrame(pestana, text=" Resultados Calculados ", padding=12)
178         frame_res.grid(row=3, column=0, columnspan=2, sticky="ew", pady=(15, 0))
179
180         lbl_res_vol = ttk.Label(frame_res, text="Volumen (V): --", font=("Arial", 11, "bold"), foreground=■ "#0056b3")
181         lbl_res_vol.pack(anchor="w", pady=4)
182         lbl_res_sup = ttk.Label(frame_res, text="Superficie Total (A): --", font=("Arial", 11, "bold"),
foreground=■ "#28a745")
183         lbl_res_sup.pack(anchor="w", pady=4)
184
185         def calcular():
186             try:
187                 r = self.obtener_valor_valido(entry_radio, "Radio")
188
189                 esfera = Esfera(r)
190                 lbl_res_vol.config(text=f"Volumen (V): {esfera.calcular_volumen():.2f} cm³")
191                 lbl_res_sup.config(text=f"Superficie Total (A): {esfera.calcular_superficie():.2f} cm²")
192             except ValueError as e:
193                 messagebox.showerror("Error de datos", str(e))
194
195         ttk.Button(pestana, text="Calcular Esfera", style="Accent.TButton", command=calcular).grid(row=2, column=0,
columnspan=2, pady=(15, 5))

```

```

196
197 # -----
198 # PESTAÑA: PIRÁMIDE CUADRADA
199 # -----
200 def crear_pestana_piramide(self):
201     pestana = ttk.Frame(self.notebook, padding=15)
202     self.notebook.add(pestana, text="Pirámide")
203
204     lbl_info = ttk.Label(pestana, text="Cálculo diseñado para una Pirámide Regular de Base Cuadrada.",
205                          font=("Arial", 9, "italic"), foreground="#666666")
206     lbl_info.grid(row=0, column=0, columnspan=2, sticky="w", pady=(0, 10))
207
208     ttk.Label(pestana, text="Lado de la Base (cm):").grid(row=1, column=0, sticky="w", pady=6)
209     entry_lado = ttk.Entry(pestana, width=18, font=("Arial", 10))
210     entry_lado.grid(row=1, column=1, pady=6, padx=10, sticky="w")
211     entry_lado.insert(0, "5.0")
212
213     ttk.Label(pestana, text="Altura (h) en cm:").grid(row=2, column=0, sticky="w", pady=6)
214     entry_altura = ttk.Entry(pestana, width=18, font=("A (parameter) sticky: str
215     entry_altura.grid(row=2, column=1, pady=6, padx=10, sticky="w")
216     entry_altura.insert(0, "10.0")
217
218     ttk.Label(pestana, text="Apotema (a) en cm:").grid(row=3, column=0, sticky="w", pady=6)
219     entry_apotema = ttk.Entry(pestana, width=18, font=("Arial", 10))
220     entry_apotema.grid(row=3, column=1, pady=6, padx=10, sticky="w")
221     entry_apotema.insert(0, "12.0")

```

```

222
223     frame_res = ttk.LabelFrame(pestana, text=" Resultados Calculados ", padding=12)
224     frame_res.grid(row=5, column=0, columnspan=2, sticky="ew", pady=(15, 0))
225
226     lbl_res_vol = ttk.Label(frame_res, text="Volumen (V): --", font=("Arial", 11, "bold"), foreground="#0056b3")
227     lbl_res_vol.pack(anchor="w", pady=4)
228     lbl_res_sup = ttk.Label(frame_res, text="Superficie Total (A): --", font=("Arial", 11, "bold"),
229     foreground="#28a745")
230     lbl_res_sup.pack(anchor="w", pady=4)
231
232     def calcular():
233         try:
234             l = self.obtener_valor_valido(entry_lado, "Lado de la Base")
235             h = self.obtener_valor_valido(entry_altura, "Altura")
236             a = self.obtener_valor_valido(entry_apotema, "Apotema")
237
238             piramide = PiramideCuadrada(l, h, a)
239             lbl_res_vol.config(text=f"Volumen (V): {piramide.calcular_volumen():.2f} cm³")
240             lbl_res_sup.config(text=f"Superficie Total (A): {piramide.calcular_superficie():.2f} cm²")
241         except ValueError as e:
242             messagebox.showerror("Error de datos", str(e))
243
244     ttk.Button(pestana, text="Calcular Pirámide", style="Accent.TButton", command=calcular).grid(row=4, column=0,
245     columnspan=2, pady=(15, 5))

```

```

244
245
246 # =====
247 # 3. HILO DE EJECUCIÓN PRINCIPAL
248 # =====
249 if __name__ == "__main__":
250     root = tk.Tk()
251     app = CalculadoraGeometricaApp(root)
252     root.mainloop()

```

Calculador Geométrico Pro

Cilindro Esfera Pirámide

Cálculo basado en el Radio de las bases y su Altura.

Radio (r) en cm:

Altura (h) en cm:

Calcular Cilindro

Resultados Calculados

Volumen (V): 2155.13 cm³

Superficie Total (A): 923.63 cm²

Calculador Geométrico Pro

Cilindro Esfera Pirámide

Cálculo simétrico basado únicamente en el Radio del cuerpo.

Radio (r) en cm:

Calcular Esfera

Resultados Calculados

Volumen (V): 904.78 cm³

Superficie Total (A): 452.39 cm²

Calculador Geométrico Pro

Cilindro Esfera Pirámide

Cálculo diseñado para una Pirámide Regular de Base Cuadrada.

Lado de la Base (cm):

Altura (h) en cm:

Apotema (a) en cm:

Calcular Pirámide

Resultados Calculados

Volumen (V): 24.00 cm³

Superficie Total (A): 69.00 cm²

